

Generator Context

Create a context manager class called `sequence_generator` that generates an infinite sequence of numbers. The sequence starts from a given value and increases by a specified step with each iteration.

The context manager should expose a `generate` method that yields numbers in the sequence.

Example

```
with sequence_generator(3, 5) as gen:
    for _ in range(3):
        print(next(gen), end=" ")
```

Output: 3 8 13

In this example, we create a sequence starting at 3 with a step of 5. We then retrieve the first three values from the sequence (3, 8, and 13) and print them.

Function Description

Complete the class `sequence_generator` in the editor below. It should be a context manager class and should expose `generate`.

`sequence_generator`'s `__init__()` method has the following parameter(s):

- `int start`: the start value
- `int step`: the increment at each call

Constraints

- $1 \leq \text{start}, \text{step} \leq 100$
- $1 \leq \text{num} \leq 10$

Input Format For Custom Testing

There are 3 lines of input, the lines contain `start`, `step`, and `num`.

Sample Case 0

Sample Input For Custom Testing

STDIN	Function
-----	-----
22	start = 22
75	step = 75
6	num = 6

Sample Output

22
97
172

```
247
322
397
```

Explanation

Six numbers are generated, starting with 22 and adding 75 at each step.

Sample Case 1

Sample Input For Custom Testing

STDIN	Function
-----	-----
45	start = 45
72	step = 72
1	num = 1

Sample Output

```
45
```

Explanation

The generator returns one number, starting with 45. If more numbers were requested, 72 would be added at each step.